

Architectural Blueprint for Multi-Tenant SaaS: Shareable Stateful Tools & Analytics

1. Executive Summary

The architectural design of a multi-tenant Software-as-a-Service (SaaS) platform demands a rigorous approach to data isolation, request routing, and component modularity. This report details the technical architecture for implementing a "Share" functionality within an existing ecosystem comprising Next.js (Frontend), Spring Boot (Backend), and PostgreSQL (Database). The specific use case—enabling public, unauthenticated access to configured financial calculators while maintaining strict tenant isolation—presents unique challenges in security filter ordering, stateless context resolution, and frontend component composition.

The proposed architecture leverages a **schema-based multi-tenancy model** where tenant context is resolved via subdomains for public links. A **Medium-style URL strategy** utilizing NanoID ensures short, URL-friendly, and collision-resistant identifiers. The backend utilizes a custom **Spring Security filter chain** configuration to permit public access to specific endpoints while enforcing tenant context prior to transaction initiation. On the frontend, the **Next.js App Router** and **React Server Components (RSC)** are utilized to decouple data fetching from presentation, allowing the reuse of complex calculator logic across both authenticated dashboard views and public share layouts. Furthermore, a robust **analytics pipeline** is designed using navigator.sendBeacon for reliable telemetry and a PostgreSQL JSONB schema for flexible event logging.

This document serves as a comprehensive implementation guide, addressing critical decisions regarding database performance, security boundaries, and code maintainability.

2. Architectural Vision and Core Patterns

2.1. The Hybrid Multi-Tenancy Model

The core of the system relies on a shared database with separate schemas per tenant, a pattern often referred to as the "Shared Database, Separate Schema" approach. This model offers an optimal balance between isolation and infrastructure cost.¹ In a standard flow, the tenant is identified via the authenticated user's credentials (JWT). However, the "Share" functionality introduces a paradigm shift: the consumer of the link is anonymous, yet the execution context must belong to a specific tenant to load the correct configurations, branding, and lead capture settings.

The architecture decouples **Tenant Identification** from **User Authentication**. While

authentication verifies *who* the user is, tenant identification determines *where* the data resides. For public share links, the tenant identity is derived strictly from the subdomain (e.g., `acme.wealth-saas.com`), whereas the user identity remains anonymous until a lead capture event occurs.¹

2.2. Stateful Configurations via Stateless Links

A key architectural requirement is the ability to share a specific *state* of a calculator (e.g., a specific investment amount, expected return rate, and tenure) without creating permanent user accounts for every visitor. This is achieved through the **Snapshot Pattern**. When a share link is generated, the current configuration of the calculator is serialized into a JSON payload and stored in a `SharedLink` entity. The unique ID of this entity becomes the key in the URL.

This transforms the URL from a mere pointer to a resource into a **State Container**. The system does not need to parse query parameters to reconstruction state; instead, it retrieves the immutable configuration snapshot from the database. This approach ensures data integrity—if the original calculator defaults change, the shared link remains faithful to the version seen by the creator.⁴

2.3. Asynchronous Telemetry Architecture

To track user engagement (time spent, unique views) without degrading user experience, the architecture employs an **asynchronous telemetry model**. The frontend utilizes the Beacon API to transmit analytics data. This non-blocking mechanism allows the browser to send data even after the page is unloaded, ensuring high-fidelity tracking of "Time on Page" metrics, which are notoriously difficult to capture with synchronous XHR or Fetch requests.⁶

3. URL Strategy and Routing Architecture

3.1. Identifier Generation Strategy

The requirement for a Medium-style URL pattern `/[tool-slug]-[unique-id]` (e.g., `sip-calculator-x8j9q2`) necessitates a unique identifier generation strategy that is both compact and secure.

3.1.1. NanoID vs. UUID and Hashids

While UUIDs (Universally Unique Identifiers) provide a standard for uniqueness, their 36-character length is unwieldy for user-facing URLs. **NanoID** is selected as the superior alternative for this architecture.⁷

Feature	UUID (v4)	NanoID	Hashids
---------	-----------	--------	---------

Length	36 characters	21 characters (configurable)	Variable (based on integer)
Alphabet	Hexadecimal (0-9, a-f)	URL-Safe (A-Za-z0-9_-)	Configurable
Collision Probability	Extremely Low	Configurable (Low)	Zero (if strictly sequential)
Predictability	Unpredictable	Unpredictable (Hardware Random)	Predictable (decodable)
Security	Secure	Secure	Insecure (Information Leakage)

- **Compactness:** NanoID allows for a configurable alphabet and length. A 21-character ID using A-Za-z0-9_- provides sufficient entropy (126 bits) to prevent collisions for trillions of IDs.⁹
- **URL Friendliness:** Unlike standard Base64 encoding, NanoID uses a URL-safe alphabet by default, eliminating the need for URL encoding/decoding steps.
- **Security:** NanoID uses a hardware random generator, making it unpredictable compared to incremental IDs or Hashids, which can be reverse-engineered to reveal the sequential count of database records.¹⁰

Design Decision: The system will generate a 12-character NanoID for public links. This length is short enough to be memorable but long enough to avoid collisions within a single tenant's namespace.

3.1.2. URL Structure

The URL structure is defined as:

`https://{tenant-subdomain}.{domain}.com/share/{tool-slug}-{nano-id}`

- **Tenant Subdomain:** Explicitly identifies the tenant (e.g., growth-investments).
- **Path Prefix /share:** Clearly distinguishes these routes from standard dashboard routes, aiding in middleware routing rules and exclusion from authentication checks.
- **Tool Slug:** Providing the tool-slug (e.g., sip-calculator) aids in SEO and user trust, giving a preview of the content.
- **Unique ID:** The identifier resolving to the specific configuration snapshot.

3.2. Next.js Middleware Routing

The Next.js Middleware acts as the ingress controller for the frontend application. It is responsible for parsing the incoming hostname to extract the tenant subdomain and rewriting the request to the appropriate internal route.¹¹

Middleware Logic Flow:

1. **Intercept Request:** The middleware intercepts every request before it hits the page rendering logic.
2. **Hostname Parsing:** It extracts the host header. If the host is app.domain.com (the main SaaS dashboard), it allows standard routing. If the host is *.domain.com, it treats the wildcard segment as the tenantId.
3. **Path Analysis:**
 - If the path starts with /share/, the middleware rewrites the URL to an internal dynamic route: /share/[tool]/[id]?tenant={subdomain}.
 - This rewrite is invisible to the user; the browser URL bar remains unchanged.
4. **Header Injection:** The middleware injects a custom header X-Tenant-ID into the request headers. This ensures that when the server-side component fetches data from the Spring Boot backend, the tenant context is explicitly propagated.¹¹

Code Structure for Middleware (middleware.ts):

TypeScript

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const hostname = request.headers.get('host') |

  | '';
  const url = request.nextUrl;

  // Extract subdomain (assuming 2-level TLD + domain)
  // e.g., tenant.app.com -> tenant
  const subdomain = hostname.split('.');
  const isMainDomain = subdomain === 'www' |

  | subdomain === 'app';

  if (!isMainDomain) {
```

```

// Inject Tenant ID for downstream use
request.headers.set('X-Tenant-ID', subdomain);

// Rewrite logic for share links
if (url.pathname.startsWith('/share')) {
  // Rewrite /share/sip-calculator-x8j9q2 to /share/[slug]/[id]
  // Requires regex parsing of the slug-id pattern
  const pathParts = url.pathname.split('/');
  const resource = pathParts[pathParts.length - 1]; // sip-calculator-x8j9q2
  const lastDashIndex = resource.lastIndexOf('-');

  if (lastDashIndex !== -1) {
    const toolSlug = resource.substring(0, lastDashIndex);
    const id = resource.substring(lastDashIndex + 1);

    // Rewrite to internal route handler
    return NextResponse.rewrite(new URL(`/public/share/${toolSlug}/${id}`, request.url));
  }
}
return NextResponse.next();
}

```

This configuration effectively segregates public traffic from authenticated application traffic at the edge, reducing latency and simplifying the logic required within the React components.¹¹

4. Spring Boot Tenant Resolution & Security Architecture

The backend architecture must solve a critical conflict: allowing public access (bypassing Spring Security authentication) while strictly enforcing multi-tenancy (requiring a tenant context for database connections).

4.1. The Filter Chain Order

In Spring Boot, the `SecurityFilterChain` typically sits early in the request processing pipeline. Standard implementations often rely on the authenticated user principal to resolve the tenant. However, for unauthenticated requests, the Principal is null.

Solution: The **Tenant Resolution Filter** must be registered *before* the Spring Security Authentication Filter.¹³ This ensures that the tenant context is established in the

TenantContext (ThreadLocal) before any security checks or controller logic execute.

Filter Order Strategy:

1. **TenantFilter:** Extracts X-Tenant-ID header or Subdomain. Sets TenantContext.
2. **Spring Security Filter Chain:** Checks if the path is permitted (public) or requires auth.
3. **Controller:** Executes business logic using the TenantContext to route database queries to the correct schema.

4.2. TenantFilter Implementation

The TenantFilter implements OncePerRequestFilter. It inspects the Origin or Host header (forwarded by Next.js middleware) to identify the tenant.

Key Logic in TenantFilter.java:

- **Extraction:** It parses the domain to extract the subdomain.
- **Validation:** It checks the extracted subdomain against a cached list of active tenants (using Redis or Caffeine for performance) to prevent database hits on every request for invalid subdomains.
- **Context Setting:** If valid, it calls TenantContext.setCurrentTenant(tenantId).
- **Cleanup:** In a finally block, it calls TenantContext.clear() to prevent memory leaks in thread-pooled environments.¹

4.3. Security Configuration for Public Endpoints

The SecurityConfig.java must explicitly permit access to the share endpoints while maintaining the filter chain integrity for other routes.

Java

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .addFilterBefore(tenantFilter, UsernamePasswordAuthenticationFilter.class) // Critical:
        // Tenant before Auth
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/public/**").permitAll() // Allow share links
            .requestMatchers("/api/analytics/**").permitAll() // Allow analytics logging
            .anyRequest().authenticated()
        )
        .sessionManagement(session ->
```

```

session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .csrf(csrf -> csrf.disable()); // Disable CSRF for stateless APIs
    return http.build();
}

```

This configuration ensures that requests to `/api/public/...` bypass the JWT check but still pass through the `TenantFilter`.¹⁵ Consequently, when the controller attempts to fetch the `SharedLink` from the database, the Hibernate/JPA layer already has the correct schema set in `TenantContext`.

4.4. Multi-Tenant Data Access

The `TenantIdentifierResolver` interacts with the `TenantContext` to supply the current tenant ID to the `Hibernate MultiTenantConnectionProvider`.

- **Schema Resolution:** The `MultiTenantConnectionProvider` uses the tenant ID to execute a `SET SEARCH_PATH TO tenant_schema` command (in PostgreSQL) on the connection.¹
- **Isolation:** This guarantees that a public request for a share link `x8j9q2` will only search the schema of the subdomain tenant. If the ID exists in another tenant's schema but not the current one, the query returns 404, preventing cross-tenant data leakage.

5. Database Design and Schema Definition

The database design utilizes PostgreSQL's robust JSONB capabilities to store flexible configuration data and distinct tables for high-volume analytics logging.

5.1. The SharedLink Entity

The `SharedLink` table stores the snapshot of the calculator configuration. Using JSONB for the configuration payload is architecturally significant because different calculators (SIP, Lumpsum, SWP) have vastly different input parameters. A rigid relational schema would require an EAV (Entity-Attribute-Value) model or many sparse columns, both of which are inefficient.¹⁸

Schema Definition:

SQL

```

CREATE TABLE shared_links (
    id VARCHAR(12) PRIMARY KEY, -- NanoID

```

```

tool_slug VARCHAR(50) NOT NULL, -- e.g., 'sip-calculator'
configuration JSONB NOT NULL, -- Snapshot of inputs { "investment": 5000, "rate": 12 }
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
expires_at TIMESTAMP WITH TIME ZONE, -- Optional expiration
is_active BOOLEAN DEFAULT TRUE,
meta_title VARCHAR(255), -- Custom OG Title for sharing
meta_description TEXT, -- Custom OG Description

-- Foreign Key to User who created it (optional, for analytics aggregation)
created_by_user_id BIGINT REFERENCES users(id)
);

-- Index for faster lookup by tool and ID
CREATE INDEX idx_shared_links_tool_id ON shared_links (tool_slug, id);
-- GIN Index for querying JSONB content if needed (e.g., find all links with >10k investment)
CREATE INDEX idx_shared_links_config ON shared_links USING GIN (configuration);

```

Architectural Insight: The JSONB column allows the frontend to pass the entire configuration state object directly to the backend. The backend validates it against a generic schema but treats it essentially as a blob. This enables the frontend team to update calculator logic and inputs without requiring database schema migrations for the SharedLink table.²⁰

5.2. The AnalyticsLog Entity

Tracking unique views and time-spent generates high-write volume. A separate AnalyticsLog table is designed to capture raw events.

Schema Definition:

SQL

```

CREATE TABLE analytics_logs (
  id BIGSERIAL PRIMARY KEY,
  shared_link_id VARCHAR(12) NOT NULL REFERENCES shared_links(id),
  session_id VARCHAR(64) NOT NULL, -- Generated by frontend per session
  event_type VARCHAR(20) NOT NULL, -- 'VIEW', 'HEARTBEAT', 'CONVERSION'

  -- JSONB for flexible event metadata
  event_data JSONB, -- { "time_spent_ms": 5000, "scroll_depth": 80 }

```



```

ip_address VARCHAR(45), -- Anonymized IP for unique visitor calculation
user_agent TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Partitioning Strategy (Optional for high scale)
-- Ideally partitioned by created_at (monthly) to manage data growth
CREATE INDEX idx_analytics_link_event ON analytics_logs (shared_link_id, event_type);

```

Optimization Strategy:

- **Write-Optimized:** This table is append-only.
- **Aggregation:** Real-time analytics queries on this table can be slow. A materialized view or a scheduled job should aggregate these logs into a link_analytics_summary table (e.g., total_views, avg_time_spent) for fast dashboard rendering.²¹

6. Frontend Reusability and Composition Strategies

A primary challenge is reusing the complex logic of calculators (SIP, SWP, etc.) residing in src/features/calculators across two distinct contexts:

1. **Dashboard:** Authenticated, editable, saves to user profile.
2. **Public Share:** Unauthenticated, read-only (initially), loads from SharedLink.

6.1. The "Smart Container / Dumb Presentation" Pattern

To maximize reuse, the components are refactored into **Presentational Components** (UI only) and **Container Components** (Logic & Data Fetching).

- **CalculatorLogic (Hook):** Contains the math, validation, and state management (e.g., useSipCalculator). This hook accepts initialValues as a parameter.²²
- **CalculatorView (Component):** A pure UI component that accepts values, onChange, and results as props. It has no internal side effects regarding data persistence.

6.2. Composition Implementation

Scenario A: Dashboard Usage

The Dashboard page wraps the CalculatorView with a DashboardContainer.

- Fetches user preferences from the API.
- Passes these as initialValues to the useSipCalculator hook.
- Injects an onSave handler that calls the authenticated POST /api/user/save endpoint.

Scenario B: Public Share Usage

The Public Share page (dynamic route) wraps the same CalculatorView with a PublicContainer.

- Fetches the configuration JSON from the SharedLink API endpoint via `getServerSideProps` or RSC data fetching.
- Passes this configuration as `initialValues` to the `useSipCalculator` hook.
- Injects a different `onAction` handler (e.g., "Download PDF") which triggers the lead capture flow instead of saving.

This pattern ensures zero code duplication for the core calculator UI and logic. Only the data injection and action handlers differ.²³

6.3. Layout Strategy using Next.js App Router

The Next.js App Router's **Route Groups** feature is utilized to apply distinct layouts without affecting the URL structure.

- `app/(dashboard)/layout.tsx`: Contains the Sidebar, User Nav, and Auth Guard.
- `app/(public)/layout.tsx`: A lightweight layout containing only the Tenant Branding (Logo, Colors) and a simplified Footer. It does *not* contain the sidebar or auth checks.

The share route resides in `app/(public)/share/[tool]/[id]/page.tsx`. This ensures that public visitors receive a focused, optimized experience without the heavy JavaScript bundles associated with the dashboard shell.²⁴

7. Analytics and Tracking Architecture

Accurate tracking of user engagement on public links is critical for the SaaS value proposition. The architecture uses a heartbeat mechanism combined with the Beacon API.

7.1. Unique Views and Session Tracking

- **Session ID:** On the client side, a `sessionId` is generated (using `NanoID`) and stored in `sessionStorage`. This persists across reloads but resets on tab close, defining a "session" accurately.
- **Unique Views:** The backend counts distinct `session_ids` per `shared_link_id` in the `AnalyticsLog`. IP address hashing is used as a secondary deduplication method if cookies/storage are blocked.

7.2. Time-Spent Tracking (Heartbeat)

Tracking "Time on Page" is implemented via a custom React hook `usePageTracking`.

Mechanism:

1. **Visibility Awareness:** The hook listens to the `document.visibilityState`. The timer pauses when the user switches tabs, ensuring only active attention time is counted.²⁶
2. **Heartbeats:** Every 15 seconds (configurable), the client sends a lightweight POST to

/api/analytics/heartbeat with the incremental time spent.

3. **Beacon API (The Safety Net):** The `beforeunload` event triggers `navigator.sendBeacon`. This guarantees that the final chunk of time (between the last heartbeat and the tab close) is transmitted. `sendBeacon` is crucial because browsers often cancel fetch requests initiated during page unload.⁶

Client-Side Implementation Snippet:

TypeScript

```
// usePageTracking.ts
useEffect(() => {
  const interval = setInterval(() => {
    if (document.visibilityState === 'visible') {
      sendHeartbeat(sessionId, timeSpent);
    }
  }, 15000);

  const handleUnload = () => {
    const payload = new Blob({ type: 'application/json' });
    navigator.sendBeacon('/api/analytics/flush', payload);
  };

  window.addEventListener('visibilitychange', handleVisibilityChange);
  window.addEventListener('beforeunload', handleUnload);

  return () => {
    clearInterval(interval);
    window.removeEventListener('visibilitychange', handleVisibilityChange);
    window.removeEventListener('beforeunload', handleUnload);
  };
},);
```

8. Lead Capture and Gated Content Flow

To convert public traffic into leads, high-value actions (e.g., "Download PDF Report") are gated behind a lead capture form.

8.1. The Flow

1. **Public Access:** User interacts with the shared calculator (ungated).
2. **Trigger:** User clicks "Download PDF".
3. **Gate:** A modal appears requesting Name, Email, and Phone.
4. **Submission:**
 - Frontend posts lead details to /api/leads (public endpoint).
 - Backend validates the tenant (via TenantFilter).
 - Backend saves the lead into the leads table associated with the tenant.
5. **Token Generation:** Upon successful lead creation, the backend generates a **Temporary Signed Token** (JWT or opaque token) valid for 5 minutes.
6. **Access:** The backend returns a signed URL: /api/downloads/report.pdf?token=xyz....²⁸
7. **Download:** The browser initiates the download using this signed URL.

8.2. Security of the Gated Asset

The download endpoint /api/downloads/{file} is protected by a specific security filter that validates the signed token. This prevents users from sharing the direct PDF link; the link expires quickly, forcing new users to go through the lead capture form to generate their own token.²⁹

9. Comprehensive Database Schema

9.1. Tables Definition

SQL

```
-- Tenant Table (Existing context)
CREATE TABLE tenants (
  id BIGSERIAL PRIMARY KEY,
  subdomain VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255),
  is_active BOOLEAN DEFAULT TRUE
);

-- Shared Links (The Configuration Snapshot)
CREATE TABLE shared_links (
  id VARCHAR(12) PRIMARY KEY, -- NanoID
  tenant_id BIGINT NOT NULL REFERENCES tenants(id),
  tool_slug VARCHAR(50) NOT NULL,
```

```

configuration JSONB NOT NULL, -- { "monthlyInvestment": 10000, "rate": 12, "years": 10 }
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
visits_count BIGINT DEFAULT 0, -- Denormalized counter for fast read
last_accessed_at TIMESTAMP WITH TIME ZONE
);

-- Analytics Log (High volume write)
CREATE TABLE analytics_log (
  id BIGSERIAL PRIMARY KEY,
  shared_link_id VARCHAR(12) NOT NULL REFERENCES shared_links(id),
  session_id VARCHAR(64) NOT NULL,
  event_type VARCHAR(20) CHECK (event_type IN ('VIEW', 'HEARTBEAT', 'DOWNLOAD_ATTEMPT',
'LEAD_CAPTURE')),
  time_spent_seconds INTEGER DEFAULT 0,
  meta_data JSONB, -- { "scroll_depth": 50, "device": "mobile" }
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Partitioning Analytics Log (Recommended for scale)
-- PARTITION BY RANGE (created_at);

-- Leads (Integration with existing table)
CREATE TABLE leads (
  id BIGSERIAL PRIMARY KEY,
  tenant_id BIGINT NOT NULL REFERENCES tenants(id),
  source_link_id VARCHAR(12) REFERENCES shared_links(id), -- Track which link generated the lead
  name VARCHAR(255),
  email VARCHAR(255),
  phone VARCHAR(50),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

```

9.2. JSONB Indexing for Analytics

To enable queries like "Find all shared links where the investment amount > 50000", a GIN (Generalized Inverted Index) is applied to the configuration column.

SQL

```
CREATE INDEX idx_shared_links_config_gin ON shared_links USING GIN (configuration);
```

This allows the analytics dashboard to filter shared links based on the *content* of the calculation, providing deep insights to the tenant (e.g., "Users sharing high-value SIP calculations convert 20% better").¹⁹

10. Conclusion and Strategic Insights

The proposed architecture successfully bridges the gap between secure, isolated multi-tenancy and open, viral public sharing.

Key Technical Wins:

1. **Security:** The combination of TenantFilter ordering and Subdomain-based resolution ensures that public endpoints are never "tenant-less," preventing data contamination.
2. **Scalability:** Using NanoID for URLs and JSONB for configuration snapshots allows the system to scale to millions of shared links without schema migrations or ID collisions.
3. **Performance:** The Next.js Middleware handling routing at the edge, coupled with RSC for data fetching, ensures fast Time-to-First-Byte (TTFB) for public visitors.
4. **Accuracy:** The dual-method tracking (Heartbeat + Beacon) solves the common problem of inaccurate "time-on-page" metrics in SPAs.

This design empowers the SaaS platform to turn every calculator usage into a potential lead generation vector, transforming static tools into dynamic, shareable assets that drive growth for tenants.

Detailed Technical Implementation Guide

1. Url Strategy: The NanoID Implementation

1.1. Why NanoID over UUID?

Standard UUIDs (e.g., 123e4567-e89b-12d3-a456-426614174000) are 36 characters long. In a "Share" context, brevity is a virtue. A shorter URL is more trustworthy and easier to share via SMS or social media.

Analysis of NanoID:

- **Alphabet:** A-Za-z0-9_- (64 characters).
- **Length:** A length of 12 characters gives 64^{12} combinations.

- **Collision Probability:** With 1,000 IDs generated per hour, it would take ~4 million years to have a 1% probability of a collision. This is more than sufficient for this application.⁸

1.2. URL Pattern Regex

The middleware must distinguish between a tool slug and the ID.

Pattern: `/[tool-slug]-[unique-id]`

Regex: `^/([a-z0-9-]+)-([A-Za-z0-9_]{10,})$`

- Group 1: Tool Slug (e.g., sip-calculator)
- Group 2: NanoID (e.g., x8j9q2z5a1)

This regex enforces that the ID must be at least 10 characters, preventing conflict with simple slugs.

2. Spring Boot Tenant Resolution: The Deep Dive

2.1. Handling Public vs. Authenticated Contexts

The challenge is that `TenantContext` usually relies on the logged-in user. For public links, we must trust the **Subdomain**.

Security Risk: A malicious actor could spoof the Host header to access another tenant's public links.

Mitigation: The `TenantFilter` must validate the subdomain against the tenants table (or cache). If the subdomain exists, the `TenantContext` is set. If not, a 404 is returned immediately, saving database resources.

2.2. The TenantFilter Code Structure

Java

```
@Component
```

```
@Order(Ordered.HIGHEST_PRECEDENCE) // Ensure this runs before Security
```

```
public class TenantFilter extends OncePerRequestFilter {
```

```
    @Autowired
```

```
    private TenantRepository tenantRepository;
```

```
    @Override
```

```
    protected void doFilterInternal(HttpServletRequest request,
```

```

        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

    String serverName = request.getServerName();
    String subdomain = extractSubdomain(serverName);

    if (subdomain != null) {
        // Check Cache first, then DB
        Optional<Tenant> tenant = tenantRepository.findBySubdomain(subdomain);

        if (tenant.isPresent()) {
            TenantContext.setCurrentTenant(tenant.get().getTenantId());
        } else {
            // Invalid tenant subdomain -> 404
            response.sendError(HttpServletResponse.SC_NOT_FOUND, "Tenant not found");
            return;
        }
    } else {
        // Handle requests to main domain (landing page) or throw error
        TenantContext.setCurrentTenant("DEFAULT_TENANT");
    }

    try {
        filterChain.doFilter(request, response);
    } finally {
        // CRITICAL: Prevent ThreadLocal leakage
        TenantContext.clear();
    }
}

```

This filter placement is non-negotiable. If placed *after* Spring Security, public endpoints configured with `permitAll()` might execute without a tenant context, leading to queries against the default schema or connection failures.¹³

3. Frontend Reusability: The Registry Pattern

To manage dozens of calculators efficiently, a **Registry Pattern** is recommended in the frontend.

3.1. Calculator Registry

Create a central registry map that links tool slugs to their Components and Configuration Schemas.

TypeScript

```
// src/features/calculators/registry.ts
import SipCalculatorView from './views/SipCalculatorView';
import LumpsumView from './views/LumpsumView';

export const CALCULATOR_REGISTRY = {
  'sip-calculator': {
    component: SipCalculatorView,
    title: 'SIP Calculator',
    schema: SipSchema // Zod schema for validation
  },
  'lumpsum-calculator': {
    component: LumpsumView,
    title: 'Lumpsum Calculator',
    schema: LumpsumSchema
  }
};
```

3.2. The Dynamic Share Page

The dynamic route `app/share/[...slug]/page.tsx` uses this registry to render the correct component dynamically.

TypeScript

```
// app/(public)/share/[...slug]/page.tsx
export default async function SharedToolPage({ params }: Props) {
  // 1. Parse URL to get ID and Tool Slug
  const { toolSlug, id } = parseParams(params.slug);

  // 2. Fetch Config from API (Server Side)
```

```

const sharedLink = await fetchSharedLink(id);

// 3. Look up component in Registry
const ToolConfig = CALCULATOR_REGISTRY;

if (!ToolConfig || !sharedLink) return <NotFound />;

const Component = ToolConfig.component;

// 4. Render with Config injected as initialValues
return (
  <PublicLayout tenant={sharedLink.tenant}>
    <Component initialValues={sharedLink.configuration} readOnly={true} />
    <LeadCaptureModal linkId={id} />
  </PublicLayout>
);
}

```

This architecture allows adding a new calculator by simply creating the View, Logic, and adding one line to the Registry. The routing and data fetching logic remain untouched.²²

4. Analytics: The Heartbeat Mechanism

4.1. Why navigator.sendBeacon?

When a user closes a tab, the browser unloads the page. Asynchronous requests (fetch, axios) initiated in the unload or beforeunload event handlers are often cancelled by the browser to preserve memory and speed up the next navigation.

navigator.sendBeacon(url, data) queues the request in the browser's networking layer. The browser guarantees to send it, even if the page context is destroyed. This is the *only* reliable way to track the final "Time Spent" metric.⁶

4.2. Handling JSON payloads in Beacon

sendBeacon sends data as POST requests. To send JSON, we must use a Blob.

JavaScript

```
const data = { sessionId: 'abc', time: 4500 };
const blob = new Blob(, { type: 'application/json' });
navigator.sendBeacon('/api/analytics/log', blob);
```

The Spring Boot backend controller must consume this application/json payload. Note that sendBeacon does not allow custom headers (like Authorization), which aligns perfectly with our public, unauthenticated endpoint strategy.³¹

5. Lead Capture: The Secure Token Handshake

To prevent users from bypassing the lead form, we use a **Temporary Access Token** pattern.

5.1. Token Generation

When the lead form is submitted to /api/leads:

1. Server validates input.
2. Server saves Lead to DB.
3. Server generates a JWT (or a random string stored in Redis) with a 5-minute expiration.
4. The token includes a claim: can_download: true.

5.2. Protected Download Endpoint

The download URL returned is /api/downloads/{fileId}?token={temp_token}.

The DownloadController checks:

1. Is the token valid?
2. Is it expired?
3. Does it grant access to *this* specific file?

If valid, the server streams the PDF file content. If not, it returns 403 Forbidden. This ensures that the PDF link cannot be copy-pasted and shared; the recipient would need to fill out the form to generate their own token.²⁹

6. Implementation Roadmap

1. Phase 1: Database & Backend Core

- Create SharedLink and AnalyticsLog tables with JSONB columns.
- Implement TenantFilter logic for subdomain extraction.
- Configure Spring Security to permit /api/public/**.

2. Phase 2: API Development

- Create SharedLinkController: Endpoints to Create (Auth) and Read (Public) links.

- Implement NanoID generation service.
- Create AnalyticsController for receiving heartbeats.
- 3. **Phase 3: Frontend Routing & Middleware**
 - Setup Next.js Middleware for subdomain rewriting.
 - Create /(public) layout group.
 - Implement the dynamic share route pattern.
- 4. **Phase 4: Component Refactoring**
 - Refactor existing calculators to accept initialValues.
 - Create the CALCULATOR_REGISTRY.
- 5. **Phase 5: Analytics & Lead Gen**
 - Implement usePageTracking hook.
 - Build LeadCaptureModal and backend token generation.
 - Integrate sendBeacon for unmount tracking.

This comprehensive plan addresses all constraints: strict multi-tenancy, secure public access, reusable code, and robust analytics.

Works cited

1. Schema-Based Multi-Tenancy with Spring Boot & Hibernate | by Oguz Topal | Medium, accessed on December 9, 2025, <https://medium.com/@oguz.topal/schema-based-multi-tenancy-with-spring-boot-hibernate-d9fb707f3603>
2. How to integrate Hibernates Multitenant feature with Spring Data JPA in a Spring Boot application, accessed on December 9, 2025, <https://spring.io/blog/2022/07/31/how-to-integrate-hibernates-multitenant-feature-with-spring-data-jpa-in-a-spring-boot-application/>
3. Deep Dive into Multitenancy Using Java Spring Boot - prama.ai, accessed on December 9, 2025, <https://prama.ai/deep-dive-into-multitenancy-using-java-spring-boot/>
4. Snapshots - Oracle, accessed on December 9, 2025, <https://www.oracle.com/webfolder/technetwork/data-quality/edqhelp/content/concepts/snapshots.htm>
5. How to build a proper DB schema to have "periodic snapshots" of a table for a selected day?, accessed on December 9, 2025, <https://stackoverflow.com/questions/15605043/how-to-build-a-proper-db-schema-to-have-periodic-snapshots-of-a-table-for-a-se>
6. Navigator: sendBeacon() method - Web APIs | MDN, accessed on December 9, 2025, <https://developer.mozilla.org/en-US/docs/Web/API/Navigator/sendBeacon>
7. Building a URL Shortener: A First System Design Test - DEV Community, accessed on December 9, 2025, <https://dev.to/shavonharrisdev/building-a-url-shortener-a-first-system-design-test-d9m>
8. Nano ID: Popular, Secure, and URL-Friendly Unique Identifiers | by Michal Gasparik, accessed on December 9, 2025,

- <https://medium.com/@gaspm/nano-id-popular-secure-and-url-friendly-unique-identifiers-1fa86c9fdf7c>
9. ai/nanoid: A tiny (118 bytes), secure, URL-friendly, unique string ID generator for JavaScript, accessed on December 9, 2025, <https://github.com/ai/nanoid>
 10. nanoid vs hashids - Awesome Elixir - LibHunt, accessed on December 9, 2025, <https://elixir.libhunt.com/compare-nanoid-vs-hashids-elixir>
 11. Subdomain-Based Routing in Next.js: A Complete Guide for Multi-Tenant Applications, accessed on December 9, 2025, <https://medium.com/@sheharyarishfaq/subdomain-based-routing-in-next-js-a-complete-guide-for-multi-tenant-applications-1576244e799a>
 12. Master Next.js Multi-Tenancy: A Deep Dive | Kite Metric, accessed on December 9, 2025, <https://kitemetric.com/blogs/mastering-deep-level-multi-tenancy-with-next-js>
 13. The Mechanics of Spring Boot's Security Filter Chain - Medium, accessed on December 9, 2025, <https://medium.com/@AlexanderObregon/the-mechanics-of-spring-boots-security-filter-chain-1fe716ba22db>
 14. Architecture :: Spring Security, accessed on December 9, 2025, <https://docs.spring.io/spring-security/reference/servlet/architecture.html>
 15. Exclude URLs from JWT security check in Spring Boot - Stack Overflow, accessed on December 9, 2025, <https://stackoverflow.com/questions/63502859/exclude-urls-from-jwt-security-check-in-spring-boot>
 16. Authorize HttpServletRequests :: Spring Security, accessed on December 9, 2025, <https://docs.spring.io/spring-security/reference/servlet/authorization/authorize-https-requests.html>
 17. Multi-tenancy Using Spring Boot - Shared Schema Subdomain Identifier Architecture, accessed on December 9, 2025, <https://techblog.geekyants.com/multi-tenancy-using-spring-boot-shared-schema-subdomain-identifier-architecture>
 18. Storing PostgreSQL JSONB Using Spring Boot and JPA - GeeksforGeeks, accessed on December 9, 2025, <https://www.geeksforgeeks.org/advance-java/storing-postgresql-jsonb-using-spring-boot-and-jpa/>
 19. JSONB: PostgreSQL's Secret Weapon for Flexible Data Modeling | by Rick Hightower, accessed on December 9, 2025, <https://medium.com/@richardhightower/jsonb-postgresqls-secret-weapon-for-flexible-data-modeling-cf2f5087168f>
 20. postgresql - Designing Tables for Large Amounts of Time-Series Data, accessed on December 9, 2025, <https://dba.stackexchange.com/questions/340189/designing-tables-for-large-amounts-of-time-series-data>
 21. PostgreSQL as a Real-Time Analytics Database - Tiger Data, accessed on December 9, 2025, <https://www.tigerdata.com/learn/real-time-analytics-in-postgres>

22. Building UI with Components - React Foundations - Next.js, accessed on December 9, 2025,
<https://nextjs.org/learn/react-foundations/building-ui-with-components>
23. How to create reusable components in Next.js - Stack Overflow, accessed on December 9, 2025,
<https://stackoverflow.com/questions/65836350/how-to-create-reusable-components-in-next-js>
24. File-system conventions: Route Groups | Next.js, accessed on December 9, 2025,
<https://nextjs.org/docs/app/api-reference/file-conventions/route-groups>
25. Getting Started: Layouts and Pages - Next.js, accessed on December 9, 2025,
<https://nextjs.org/docs/app/getting-started/layouts-and-pages>
26. Exploring Document Visibility in React with the useDocumentVisibility Hook - Medium, accessed on December 9, 2025,
<https://medium.com/@josephat94/exploring-document-visibility-in-react-with-the-usedocumentvisibility-hook-9ecdd134d401>
27. Track Page Metrics Easily with the Beacon API - SajanTech's, accessed on December 9, 2025,
<https://sajankumarv.tech/p/track-page-metrics-easily-with-the-beacon-api>
28. Signed URL Generation in Spring Boot for Private File Access - Medium, accessed on December 9, 2025,
<https://medium.com/@AlexanderObregon/signed-url-generation-in-spring-boot-for-private-file-access-65ae77515e99>
29. Making a Secure Download Link System in Spring Boot | Medium, accessed on December 9, 2025,
<https://medium.com/@AlexanderObregon/making-a-secure-download-link-system-with-spring-boot-c92796cd46b9>
30. How do you structure your project files when using App Router? : r/nextjs - Reddit, accessed on December 9, 2025,
https://www.reddit.com/r/nextjs/comments/1jfr4q0/how_do_you_structure_your_project_files_when/
31. Navigator.sendBeacon() to pass header information - Stack Overflow, accessed on December 9, 2025,
<https://stackoverflow.com/questions/40523469/navigator-sendbeacon-to-pass-header-information>
32. Offloading File Transfers with Amazon S3 Presigned URLs in Spring Boot - Reflectoring, accessed on December 9, 2025,
<https://reflectoring.io/offloading-file-transfers-with-amazon-s3-presigned-urls-in-spring-boot/>